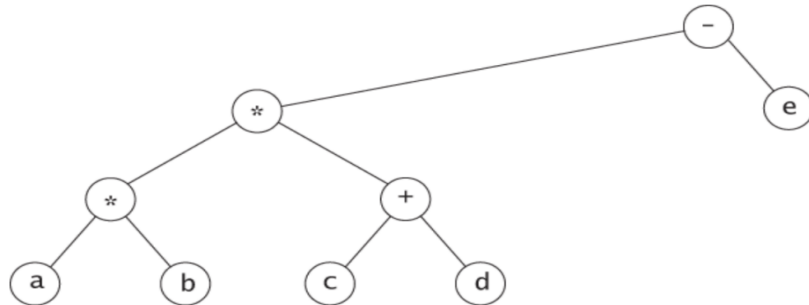


Final Exam

Banoth Jeevan Kumar

1. Give the prefix, infix, and postfix expressions that represent the tree below: **(15 points)**
(Please pay attention to the precedence of math operations and add parentheses when needed)



- a) Prefix:
- b) Infix:
- c) Postfix:

Ans: -

To give the prefix, postfix and infix expression of the given tree we need to first perform depth-first traversal of the tree, this determines the expression type.

In the shown figure, “-” is the subtraction, “*” represents the multiplication process, “+” is for the addition operation, and the leaf nodes are a, b, c, d and e.

Now let's find out prefix, postfix and infix expressions of the given Figure.

a) Prefix: -

- In this expression, the operator comes before its operands.
- The prefix expression of the given tree is: $-** (ab) + (cd) e$

{To avoid any type of confusion: i.e., $-**ab+cde$ }

b) Postfix: -

- In the postfix expression the operator comes after the operands.
- The postfix expression of the given tree is $(ab) * (cd) + * e -$.

c) Infix: -

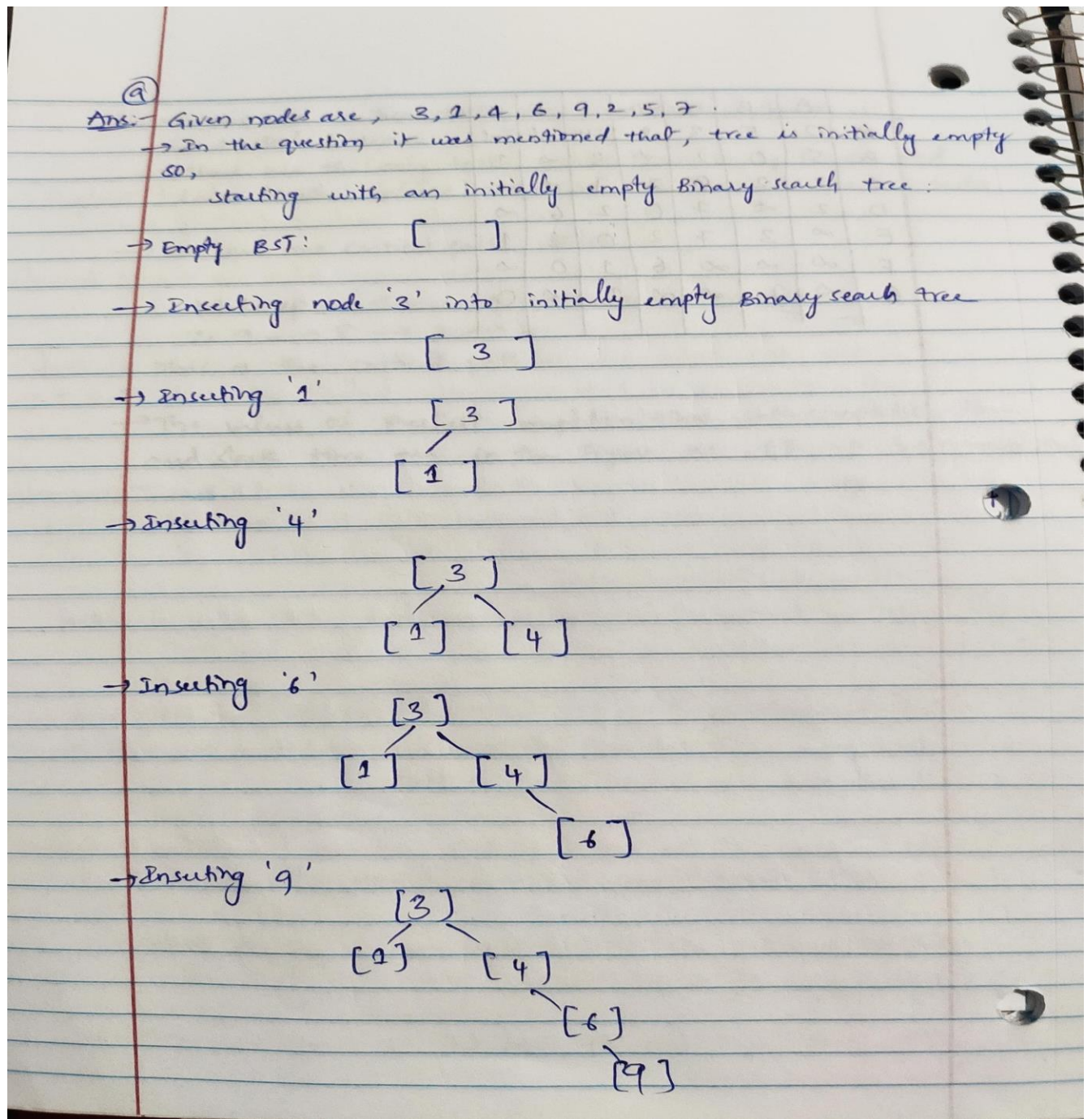
- In the infix expression the operator is placed in between the operands.
- The infix expression of the given tree is: $(a*b) * (c+d) - e$.

2. For each of the following cases, draw the result binary search tree: (10 points)

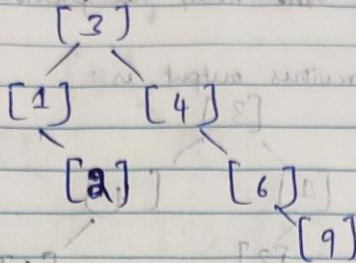
a. After inserting 3, 1, 4, 6, 9, 2, 5, 7 into an initially empty binary search tree.

b. After deleting the root from the binary search tree of part a.

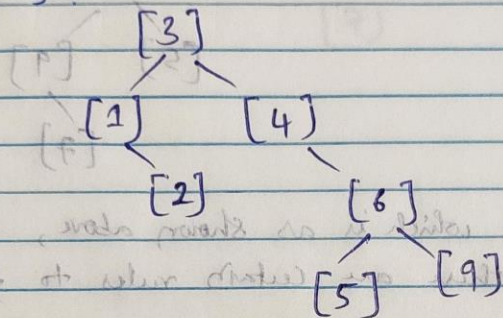
Ans:



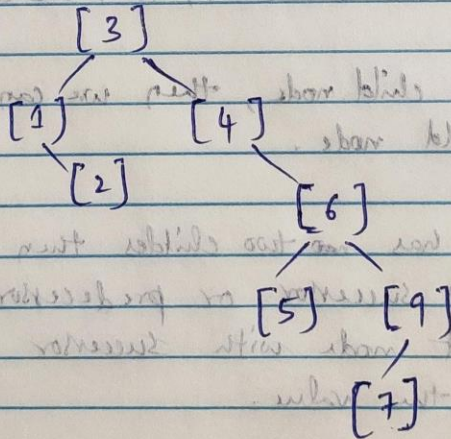
→ Inserting '2'



→ Inserting '5'



→ Inserting '7'

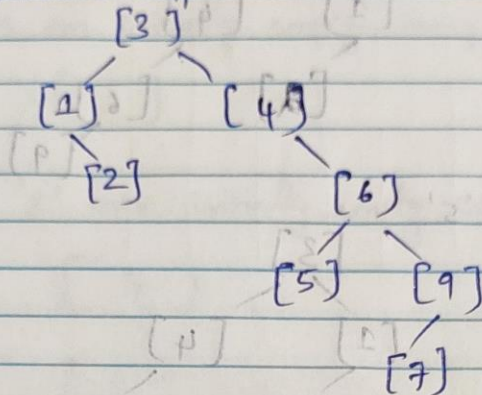


→ This is the final binary search tree which is formed from the given input nodes;

#

⑥. Deleting the root node from the previously formed tree.

→ The tree from previous output is:



→ From the tree which is as shown above, for removing the root node, there are certain rules to follow

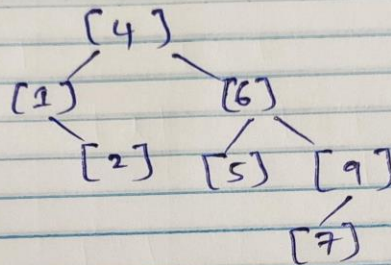
→ If the node has no children, then we can simply remove it.

→ If it has a child node, then we can replace the node with its child node.

→ If the node has two children then we need to find the in-order successor or predecessor and need to replace that node with successor or predecessor depending on the value.

→ Here, after deleting node (3) which is root node and has two children, and the in-order successor for it is (4) then we need to replace that node after the root node is being deleted.

→ The tree after removing the root node is as shown as below:



3. Draw all the rotations that you must perform and the final AVL tree after the following elements are inserted in the given order starting from an empty tree: 1, 10, 5, 7, 3, 13, 6, 4, 8, 9.

Ans: -

③
Ans:- constructing an AVL tree with given elements by graphical representation: 1, 10, 5, 7, 3, 13, 6, 4, 8, 9.

→ initially tree is empty so, []

→ inserting '1' [1]

→ inserting '10' [10] [2]
[1] [10] [2]

→ inserting '5' [1] [10] [2]
[1] [10] [2]
[5] [10] [2]

→ performing right rotation (1 pivot)

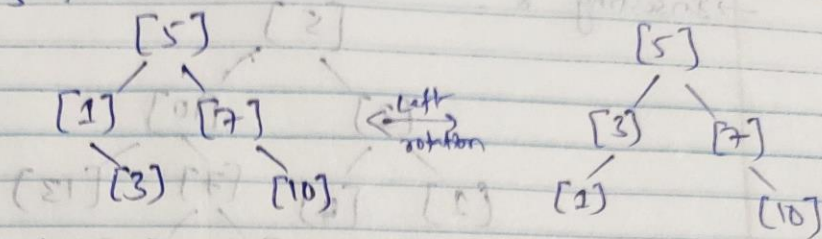
[5] [10] [2]
[1] [10] [2]
[5] [10] [2]

→ inserting '7' [5] [10] [2]
[1] [10] [2]
[5] [10] [2]

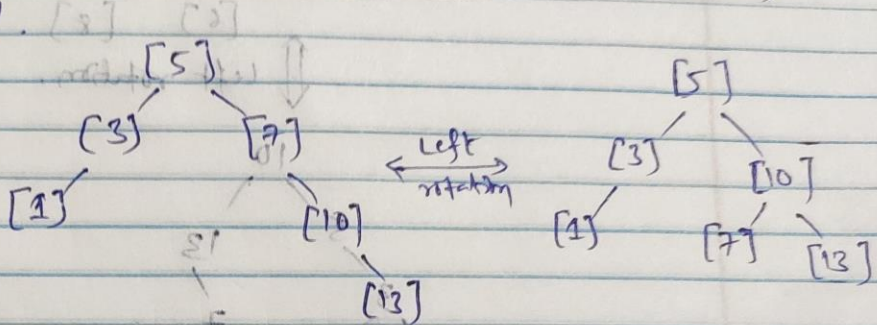
→ performing right rotation at '10'

[5] [10] [2]
[1] [10] [2]
[5] [10] [2]
[1] [10] [2]

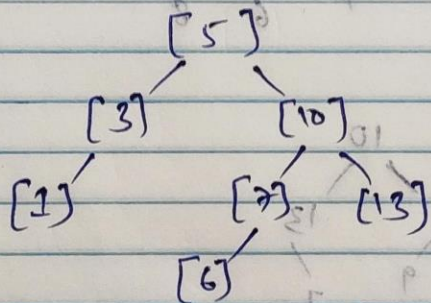
→ inserting '3'



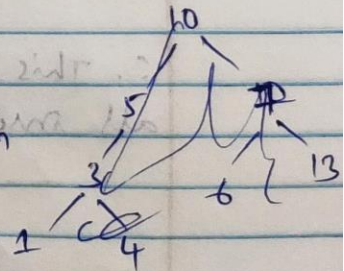
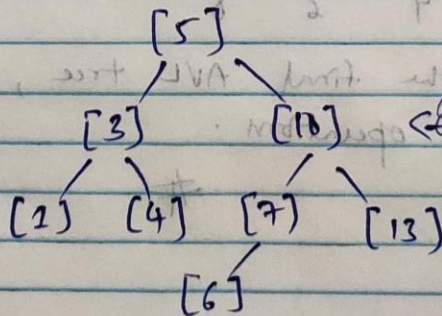
→ inserting '13'



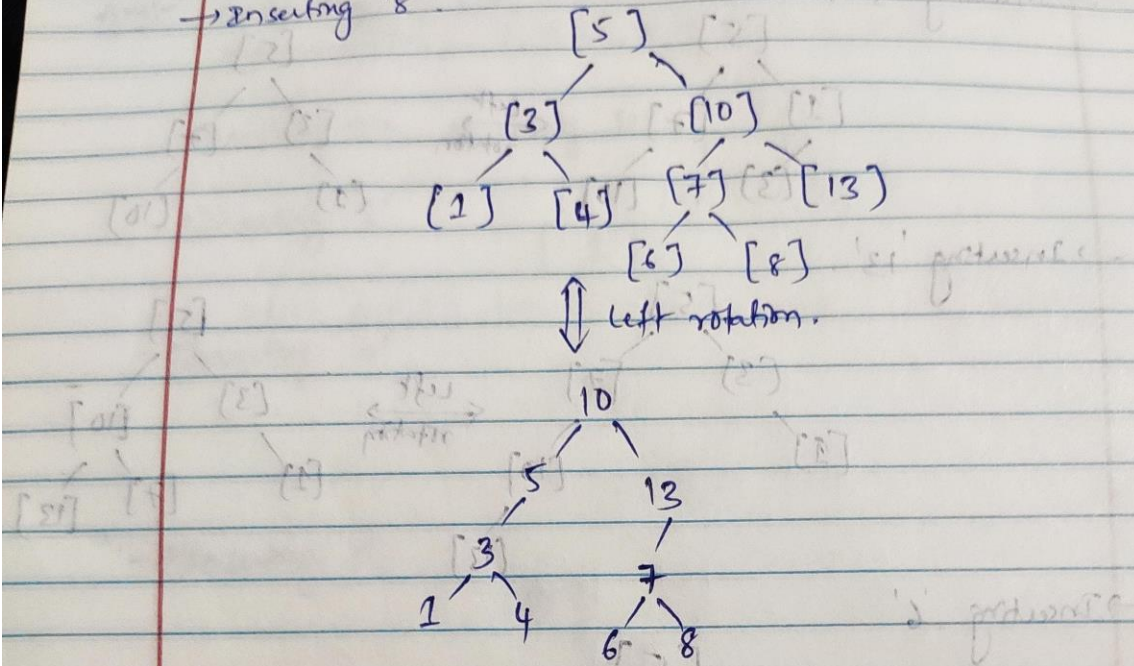
→ inserting '6'



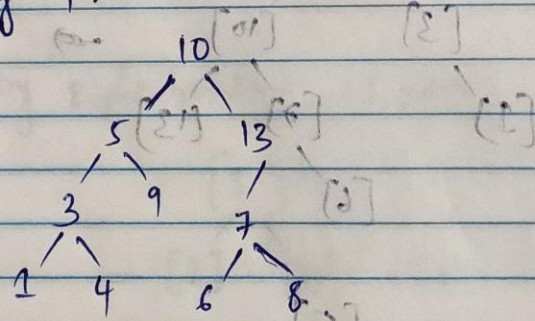
→ inserting '4'



→ Inserting '8'.



→ Inserting '9'.



∴ This is the final AVL tree, after performing the all insertion operations.

4. Given the following activities/tasks and their durations, draw an **activity-node graph** (not the event-node graph) from the **wake up** to the **leave** activities/tasks. (10 points)

Task Number	Task Name	Duration	Depends On
1	Choose clothes	3	9
2	Get Dressed	6.5	1, 8
3	Eat breakfast	6	5, 6, 7
4	Leave	1	2, 3
5	Make coffee	4.5	9
6	Make toast	2.0	9
7	Pour juice	0.5	9
8	Shower	8.5	9
9	Wake up	0	-

4). Given Tasks and their durations, for that we only need to draw activity node graph.

Ans:-

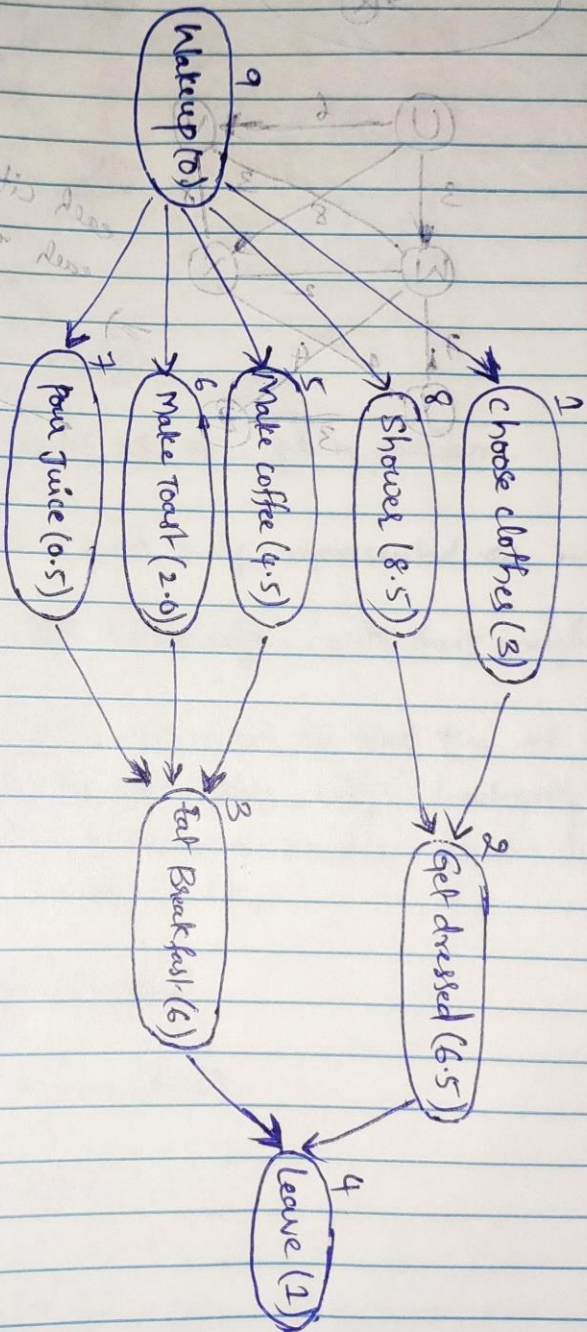


fig: Activity-node Graph.

→ thus we have only one node at starting and ending so there's no need of adding the start and end nodes.

5. There are 6 cities: U, V, W, X, Y and Z. The cost of all possible routes between each pair of cities is shown in the table below. Note that " ∞ " means that there is no direct path between the two cities (cost is ∞). Your task is to determine a set of routes that will minimize the cost of connecting city Z to all other cities (not necessarily direct connections). **(15 points)**

	U	V	W	X	Y	Z
U	0	6	3	8	∞	∞
V	6	0	3	8	∞	∞
W	3	3	0	2	7	4
X	8	8	2	0	4	∞
Y	∞	∞	7	4	0	3
Z	∞	∞	4	∞	3	0

- Map the problem into a graph model (i.e. create a graph): what does a city represent?
What does a route represent?
- Describe an algorithm to achieve your task.
- Show how the algorithm works for this specific set of computers and costs. Please show your detailed work.

5.

Ans:

From the given matrix model, the equivalent graph model is shown below.

a)

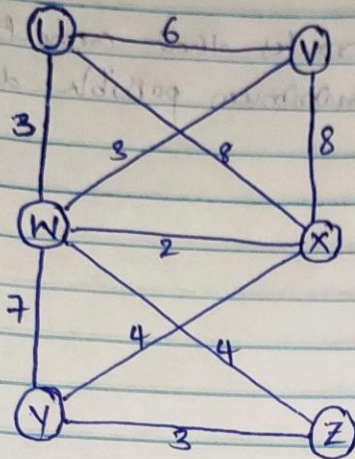


fig:- Graph model of the given problem.

6.

→ In the graph model, each city represented as node,

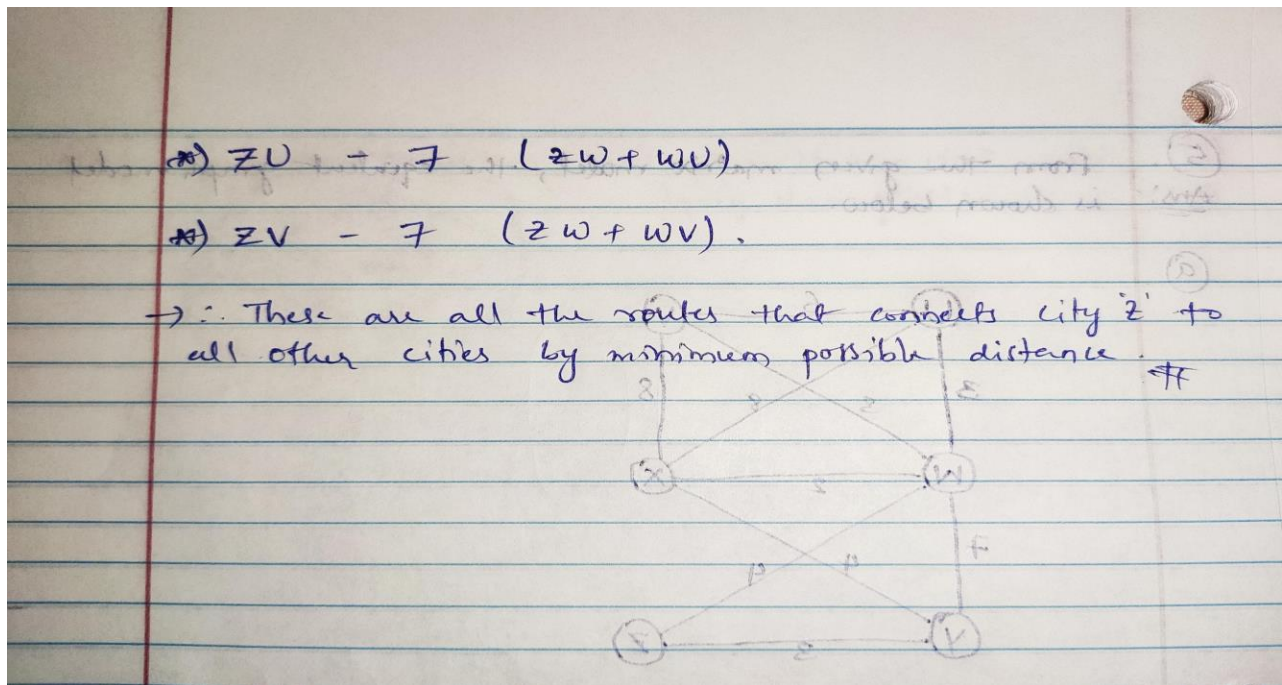
→ Each route represented as a edge. (with their weight)

* In the question given that, we need to find the set of routes that connects city 'z' to all other cities' (not only direct connect)
→ we need to find the minimum routes that connects 'z' from all other nodes so,

* ZY - 3 (direct connection)

* ZW - 4 (direct connection)

* ZX - 6 (ZW + WX)



b) Algorithm:

To minimize the cost of connecting city Z to all other cities, we can use Dijkstra's algorithm, a single-source shortest path algorithm. The algorithm works as follows:

- Initialize the distance to city Z as 0 and the distance to all other cities as infinity.
- Create a priority queue (min-heap) to store the cities ordered by their distances from Z.
- Start with Z, set its distance to 0, and insert it into the priority queue.
- While the priority queue is not empty:
 - Remove the city with the minimum distance from the priority queue.
 - For each neighboring city (connected by a direct route), calculate the potential distance from Z through the current city.
 - If the potential distance is less than the recorded distance for that city, update the distance and add the city to the priority queue.
- After the algorithm is completed, the distances to all cities from Z will be minimized.
- Once we have found the cheapest routes to all other cities, we can build a minimum spanning tree (MST) of the graph. The MST is a subset of the

edges in the graph that connects all the vertices in the graph at the lowest possible cost.

- The cost of the MST is the minimum cost of connecting city Z to all other cities.

C) Detailed work for the specific set of computers and costs

To find the cheapest route from Z to each of the other cities We can use Dijkstra's algorithm

- ☐ Initialize a set S of visited vertices to be empty.
- ☐ Initialize a priority queue Q, where the priority of a vertex is its distance from the source vertex Z.
- ☐ Add the source vertex Z to Q.
- ☐ While Q is not empty:
 - Remove the vertex u with the highest priority from Q.
 - Add u to S.
 - For each neighbor v of u that is not in S:
 - If the distance from u to v is less than the distance from the source vertex Z to v, then update the distance from Z to v to be the distance from u to v.
 - Add v to Q with the updated distance.
- ☐ The distance from the source vertex Z to each vertex in the graph is now the shortest distance between them.

Example:

Initialize Q:

Vertex | Distance

-----|-----

Z | 0

U | ∞

V | ∞

W | ∞

X | ∞

Y | ∞

While Q is not empty:

Remove Z from Q and add it to S.

Update the distance from Z to U to be 7. ($ZW+WU$)

Add U to Q with the updated distance. Remove U from Q and add it to S.

Update the distance from Z to V to be 7. ($ZW+WV$)

Add V to Q with the updated distance. Remove V from Q and add it to S.

Update the distance from Z to W to be 4. (Direct)

Add W to Q with the updated distance. Remove W from Q and add it to S.

Update the distance from Z to X to be 6. ($ZW+WX$)

Add X to Q with the updated distance. Remove X from Q and add it to S.

Update the distance from Z to Y to be 3. (Direct connection)

Add Z to Q with the updated distance. Remove Z from Q and add it to S.

Q is now empty.

The distance from Z to each of the other cities is now the shortest distance between them:

City | Distance

-----|-----

U | 7

V | 7

W | 4

X | 6

Y | 3